

```
-- single line comment
/*
multi
line
comments
*/
```

```
/*sql:-
(structured query language)
is a language that is used to communicate with databases.
Using sql you can
- create databases
- Create tables
- insert data
- update data
- delete data
- retrieve data
```

- Type of SQL Commands

1) DDL(Data definition language):-

DDL commands are used to define database structure.

- create
- alter
- drop
- truncate
- rename

example : create database databaseName;

2) DML(Data manipulation language):-

DML is used to modify data inside our tables

- insert
- update
- delete
- select

3) DQL(Data query language):-

used to fetch data

- Select

4) DCL(Data control language):-

used to control access/permissions

- grant
- revoke

5) TCL(Transaction control language):-

used to manage transactions

- commit
- rollback
- savepoint

```

*/

-- ddl commands
-- create :-
-- how to create database
create database testDB3;
create database if not exists testDB3;

-- how to use a database
use TESTDB3;

-- HOW TO CREATE TABLES
CREATE TABLE TABLE_NAME(COLUMN1 INT, COLUMN2 VARCHAR(225));
CREATE TABLE EMPLOYEES(E_Id INT, E_NAME varchar(225));

-- HOW TO SEE THE STRUCTURE OF YOUR TABLE
DESC TABLE_NAME;
DESCRIBE TABLE_NAME;

-- how to add a column to your table.
alter table employees add column salary decimal(9,2);
desc employees;
-- how to change a column name
alter table employees rename column salary to e_salary;

-- how to modify a columns data type
alter table employees modify e_salary varchar(225);

-- drop
-- how to drop a database
drop database testdb;
drop database testdb2;

-- how to drop a table
drop table if exists table_name;

--
desc employees;
-- how to drop a column from a table.
alter table employees drop column e_salary;

-- truncate:-
-- There are two to delete all the records from your table delete/ truncate
-- truncate is used to delete all the records from your table at once.
-- truncate is faster than delete.
-- truncate can't be rolled back.

```

```
insert into employees values(1,"Ray"),(2,"Shree"),(3,"Jay");
select * from employee_table;
desc employees;
```

```
truncate employees;
delete from employees;
```

```
-- Rename:-
-- how to rename a table
rename table employees to employee_table;
-- how to rename a column
alter table employee_table rename column e_id to emp_id;
desc employee_table;
```

```
/*
```

```
2) DML(Data manipulation language):-
```

```
DML is used to modify data inside our tables
```

```
- insert
- update
- delete
- select
```

```
*/
```

```
-- 1st Insert:-
```

```
-- 1st step
```

```
create database if not exists session1;
```

```
-- 2nd step
```

```
use session1;
```

```
-- 3rd step
```

```
create table employees (emp_id int, emp_name varchar(225), emp_age int);
```

```
-- 4th step
```

```
insert into employees values(101,"Raj",25);
```

```
insert into employees values
```

```
(102,"Sham",34),
```

```
(103,"Sonal",23),
```

```
(104,"Paras",45);
```

```
insert into employees values
```

```
(105,"Kiran",23),
```

```
(106,"Ravi",null);
```

```
insert into employees
```

```
(emp_name, emp_id) values
```

```
("Tisha",107);
```

```
insert into employees
```

```

(emp_age, emp_id, emp_name) values
(24,108,"Sahil"),
(34,104,"Kiran");
-- 5th step
select * from employees;
desc employees;
-----

-- 2) Update:-
select * from employees;

update employees
set emp_age = 30
where emp_id = 106;

-----

-- 3) delete:-
delete from employees where emp_id = 108;
select * from employees;
-----

-- Data types:-
-- 1) Numeric data types:-
-- 2) String data types:-
-- 3) Date and time data types:-
-- 4) Binary data types:-

-- 1) Numeric data types:-
-- Used to store numeric values:-
-- a) Integer types:-(stores whole numbers(no decimals))
-- Type          storage          range
-- tinyint       1 byte           -128 to 127   --> 0 - 255 unsigned
-- smallint      2 bytes          -32k to 32k
-- mediumint     3 bytes          -8m to 8m
-- int/Integer   4 bytes           -2B to 2B
-- bigint        8 bytes          very large number

create table testTable(testId tinyint);
create table testTable(testId tinyint unsigned);
insert into testTable values(255);
select * from testTable;

create table employees(emp_id int, emp_phone_number int);
alter table employees modify column emp_phone_number bigint;
insert into employees values(101,1234567890);
select * from employees;
insert into employees values(102,9816623423);
insert into employees values(102.967,9816623423);

```

```

-- Decimal/floating point numbers
-- used for numbers with decimals
-- 1) Decimal(Most important for money):-
desc employees;
alter table employees add column salary decimal(9,2);

select * from employees;
update employees set salary = 95000000.32 where emp_id = 103; -- 2.132 --> 2 --> 3

-- 2) Float and double

alter table employees modify column attendance double;

select * from employees;

update employees set attendance = 78.32423523432423324324
where emp_id = 103;

show variables like "%safe%";

set sql_safe_updates = off;
set sql_safe_updates = 0;
set sql_safe_updates = on;
set sql_safe_updates = 1;

-- -----
-- string data types
-- Char:-
-- is stores fixed length strings and occupy all the data that has been allocated
to it:-
create table students(s_id int, sname char(10));

insert into students values(1, "Karan");

-- varchar(length):-
-- varchar is used to store strings of variable length
alter table students modify column sname varchar(225);
insert into students values(102, "Shree");
desc students;

-- text:-
-- used for large text
-- types of text :- tinytext 255 char, text 65k char, mediumtext 16mb, longtext 4gb

-- enum :-
-- enum stores one value from a predefined list.
-- enum ensures valid values are inserted

```

```
create table employees2 (emp_name varchar(225), emp_degree
enum("BTech","BBA","MTech","MBA"));
insert into employees2 values("kashish","btech,mba");
select * from employees2;
```

```
-- set :-
-- set stores multiple values from a predefined list
create table employees2(emp_name varchar(225), emp_skills
set("Java","python","SQL","AWS","JS"));
insert into employees2 values("Ron", "java,sql,aws,Ruby");

select * from employees2;
```

```
-- date and time data types:-
```

```
-- -----
-- create a database
-- name the database session1
-- use the database
-- create a table "Employees" with columns emp_id int, emp_name varchar(225),
emp_salary decimal(9,2),
-- emp_attendance float, emp_cgpa float.
-- modify column emp_cgpa data type to double
-- change column emp_attendance to emp_attd
-- rename table employees to employees_demo
-- insert 5 records to the table
-- update 1 record and set their salary to 90000
-- delete 1st employee from your table
```

```
use session1;
/* Data types
numeric data types
```

String data types
date and time
binary
*/

-- date and time data types:-

-- date, time ,datetime, timestamp, year

-- data type	format	range
-- date	YYYY-MM-DD	1000-01-01 TO 9999-12-31
-- TIME	HH:MM:SS	00:00:00 TO 23:59:59
-- DATETIME	YYYY-MM-DD HH:MM:SS	1000-01-01 00:00:00 TO 9999-12-31 23:59:59
-- TIMESTAMP	YYYY-MM-DD HH:MM:SS	1970-01-01 TO 2038-01-19
-- YEAR	YYYY	1901 TO 2155

-- 1) date

create table students2(s_id int, s_name varchar(225),
dob date);

insert into students2 values(101, "Ravi","2005-09-09");

insert into students2 values(102, "shree",curdate(),curtime());

select * from students2;

-- time

-- add a new column name it student login_time

alter table students2 add column login_time time;

desc students2;

update students2

set login_time = "10:43:55"

where s_id = 101;

select * from students2;

-- datetime

create table orders(order_id int,
order_time datetime);

insert into orders values
(1001,"2026-02-24 22:00:00");

insert into orders values

(1002,now());

select * from orders;

-- timestamp

-- stored date and time just like datetime

-- but automatically converts based on time zone

```

-- default value can be current time

create table users(user_id int, user_name varchar(225),
created_at timestamp);
insert into users values(101,"Ron",current_timestamp());

select * from users;

-- year
create table movies(movie_id int, movie_name varchar(225),
relase_year year);
desc movies;

insert into movies values(10001,"Sholay","1972");
select * from movies;

-- day, month, week, hours, seconds, minutes etc

-- Binary data types:-
-- Binary data types store raw bytes(0s and 1s) instead of human redable
-- characters.
-- used for images, documents, pdfs, audio etc.

-- blobs(binary large objects):-
-- tinyblob - 255bytes
-- blob - 65kb
-- mediumblob - 16mb
-- largeblob - 4gb

-- Fixed/variable binary strings
-- binary
-- varbinary

create table products(product_id int,
product_name varchar(225), product_image longblob);
-- wrong way
insert into products
values(101,"amazon alexa","C:\Users\wwwva\Pictures\images.jfif");

insert into products
values(101,"amazon alexa",
load_file("C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/images.jfif"));
truncate products;

alter table products modify column product_image tinyblob;
desc products;
select * from products;

-- binary and varbinary

```

```

-- 1) Binary :- fixed length binary data
create table users2(id int, secret_code binary(5));
insert into users2 values(101, "Abc");
alter table users2 modify column secret_code varbinary(5);

select * from users2;

-- create a database and name it session2
-- use that database
-- create two tables
-- users table :- user_id int, user_name varchar(225), age int,
-- dob date, login_time time, created_at timestamp, user_image longblob

-- products table :- product_id int, product_name varchar(225), price double,
-- listed_at datetime, launch_year year, product_image longblob

-- insert 5 users and 5 products
-- alter user table and add user_city varchar(225)
-- alter products table and rename price to product_price
-- alter table products and rename it to amazon_products
-- alter table products and modify price datatype to decimal(9,2)

-- constraints:-
-- constraints are rules applied on table columns to ensure data accuracy,
-- data consistency, data integrity and bussiness rule enforcement.

-- not null, unique, primary key, foreign key, check, default.

use session1;
-- 1) Not null:-
-- not null means a column can't store null values
create table employees (emp_id int,
emp_name varchar(225) not null);
insert into employees (emp_id) values (101); -- wrong
insert into employees values(101,"Vinay"); -- right way
select * from employees;

-- how to see constraints on a table
desc employees;

-- 2) Unique constraint:-
-- unique constraint enforces unqiuness of a column.
-- Ensures all values in a column are different.
create table employees (emp_id int,
emp_name varchar(225) not null,

```

```

emp_email varchar(225) unique);

desc employees;
insert into employees values(101,"Shree","shree@gmail.com"),
(102,"Shree","shree1@gmail.com");

select * from employees;

-- 2nd way to add unique constraint
-- how to give a constraint a name
create table employees (emp_id int,
emp_name varchar(225) not null,
emp_email varchar(225),
constraint UQ_emp_email unique(emp_email));

desc employees;
-- how to drop a constraint
alter table employees drop constraint UQ_emp_email;

-- how to add a constraint after you have already created a table
alter table employees add constraint UQ_emp_email unique(emp_email);

-- 3) Primary key:-
-- Primary key is not null and unique by default.
-- Each table can have only one primary key.

create table employees (emp_id int primary key,
emp_name varchar(225) not null,
emp_email varchar(225));

insert into employees values(101,"Shree","s@gmail.com"),
(102,"Sai","sai@gmail.com"),
(103,"Priya","p@gmail.com");

select * from employees;
desc employees;

drop table employees;

-- 4) Foreign key:-
-- Foreign key maintains relationship between two tables.
-- child table column references
-- parent table primary key
-- Foreign key is used to maintain referential integrity between parent and
-- child table.

-- Rules:- 1) Parent column must be either primary key or unique.
-- 2) Data type must match.

```

```

-- 3) Engine must be InnoDB.

-- product table(Referenced table)
create table product_table(prod_no int primary key,
prod_name varchar(225) not null,
qty int);
-- sales_table(Referencing table)
create table sales(invoice_no int primary key,
product_number int,
qty int,
constraint FK_product_sales foreign key(product_number)
references product_table(prod_no));

insert into product_table
values(210,"curtain",330),
(311,"carpet",250),
(410,"office tray",250),
(420,"office chair",135);

insert into sales
values(311,410,500),
(312,420,20),
(313,210,35),
(314,311,40),
(315,410,10);

insert into sales
values(316,311,10);

select * from product_table;
select * from sales;

delete from sales where invoice_no in (314,316);

delete from product_table
where prod_no = 311;

-- On delete cascade and on update cascade

drop table product_table;
drop table sales;

create table prod_table(
prod_no int primary key,
prod_name varchar(225) not null,
qty int
);
create table sales(
invoice_no int primary key,

```

```
prod_no int,  
qty int,  
constraint FK_product_sales foreign key(prod_no)  
references prod_table(prod_no) on delete cascade on update cascade  
);
```

```
insert into prod_table  
values(210,"curtain",330),  
(311,"carpet",250),  
(410,"office tray",250),  
(420,"office chair",135);
```

```
insert into sales  
values(311,410,500),  
(312,420,20),  
(313,210,35),  
(314,311,40),  
(315,410,10);
```

```
select * from prod_table;  
select * from sales;
```

```
update prod_table  
set prod_no = 450  
where prod_no = 420;
```

```
delete from prod_table where prod_no = 410;
```

```
-- 5) default:-  
-- default is used to take a default value by default if there is not  
-- value presented at the time of insertion.  
create table employees102(emp_id int primary key,  
emp_name varchar(225),  
emp_location varchar(225) default "Pune");
```

```
insert into employees102 values(101,"Ram",default);  
insert into employees102 values(102,"Ram",null);  
insert into employees102 (emp_id, emp_name) values(103,"Sham");
```

```
select * from employees102;
```

```
create table products101(prod_id int primary key,  
prod_name varchar(225),  
listed_at datetime default now());
```

```
insert into products101 values(101,"Stuff toy",default);
```

```
select * from products101;
```

```
-- 6) Check:-
```

```
-- Ensures a condition is true before inserting it.
```

```
create table club_members(member_id int primary key,  
name varchar(225) not null,  
age int check(age >= 21));
```

```
insert into club_members values(102,"sonal",21);
```

```
select * from club_members;
```

```
-- auto_increment:-
```

```
create table club_members(  
member_id int primary key auto_increment,  
name varchar(225) not null,  
age int check(age >= 21)) auto_increment = 100;
```

```
-- auto_increment:-
```

```
create table club_members(  
member_id int primary key auto_increment,  
name varchar(225) not null,  
age int check(age >= 21));
```

```
desc club_members;
```

```
alter table club_members auto_increment = 101;
```

```
insert into club_members (name,age)values("Hritik",45),  
("Sanjay",32),  
("Kiran",34),  
("Rita",24);
```

```
select * from club_members;  
drop table club_members;
```

```
-- foreign key:-
```

```
-- employees --> managers
```

```
-- Referenced(managers) -- Pk
```

```
-- Referencing(employees) -- FK
```

```
create table managers(m_id int primary key auto_increment,
```

```
m_name varchar(225) not null) auto_increment = 101;

create table employees(e_id int primary key auto_increment,
e_name varchar(225) not null,
manager_id int,
foreign key(manager_id) references managers(m_id)) auto_increment = 1001;

insert into managers (m_name)values("Ravi"),("Sunil");

insert into employees (e_name,manager_id)
values("Sonal",101),
("Krish",102),
("Rohan",101);

select* from managers;
select * from employees;

drop table employees;

alter table employees auto_increment = 1001;
```